Session 5 Lab on Persistence and Relationships

Lebanese University | Master 1 Computer Science

Develop the same Task Manager from start to finish

Lina first needs tasks that survive a server restart. As her work grows, she groups those same tasks into projects, adds project details and shares tags. You will implement these requirements in one Flask application and keep the original tasks throughout.

Use extracted folder, task_api package and working database at instance/task_manager.db. Complete CRUD first. Then extend the same models, serializer and store with relationships. No earlier lab files are required. The Session 4 presentation is your starting reference.

The cumulative learning path

Part	You add	Schema version
A	Task model, representation and database CRUD	1
B	Project model, task assignment and navigation	2
C	One optional details record per project	3
D	Shared tags and an association table	4
E	Related reports and one transaction for several objects	4

How to work

Complete exercises in order and run each named check. If working in pairs, switch the driver at each checkpoint. Both partners should explain the code. Record returned ids and request results in EVIDENCE_TEMPLATE.md. Keep T1 and T2 from part A for every later part.

The supplied HTTP routes, validation, error handling, request client and schema commands provide the infrastructure. You write the persistence methods and add relationship mappings when each requirement appears. CHECKPOINTS.md lists the checks for each step.

1 Set up the project

Extract IN432_Session5_Student_Starter.zip. Open the folder containing requirements.txt, task_api, payloads, checks.py and README.md. All commands in this lab run from this same folder. Use Python 3.10 or later.

Windows Command Prompt

```
python --version
python -m venv .venv
.venv\Scripts\activate.bat
python -m pip install -r requirements.txt
```

macOS or Linux

```
python3 --version
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Two terminals for HTTP work

Terminal A will run the Flask server. Terminal B sends requests. Both must use this same project folder and activated environment. In each new Windows Command Prompt run .venv\Scripts\activate.bat; on macOS or Linux run source .venv/bin/activate.

Wait until A1 is complete before initializing the database. An unfinished method produces HTTP 501 with its exercise number. A later model route can also return 501 until the model has been added. A connection error means the server cannot be reached; it is a different issue.

2 Locate the code and read the Python

File	Responsibility
task_api/__init__.py	Create and configure Flask; provide CLI commands
task_api/extensions.py	Create the shared db extension and enable SQLite foreign keys
task_api/models.py	Map classes to tables and build public dictionaries
task_api/store.py	Read and change database objects
task_api/routes.py	Receive requests and choose response status codes
task_api/validation.py	Validate the whole body before calling the store
task_api/schema.py	Supplied incremental changes to the working SQLite schema
checks.py and api_client.py	Isolated checks and the HTTP request client

Keep the same files for all parts. Work in the marked regions. For C1, add the details relationship inside the Project class that you create in B1. Keep class names, method signatures, supplied imports and indentation.

Python reminders

class Task(db.Model) uses inheritance, like Java extends. def declares a method; self refers to the receiving object, like Java this. A colon introduces an indented block. Use four spaces per level. from .extensions import db imports from the same package.

```
# Generic dictionary example
settings = {"visible": False}
if "visible" in settings:
    print(settings["visible"])
```

The key exists, so the print runs even though its value is False. Dictionaries map keys to values. Lists and relationship collections hold items; append adds an item and remove removes a known item. Use True, False and None in Python; use true, false and null in JSON. PYTHON_GUIDE.md provides more examples.

Question 1. Trace POST /api/tasks in routes.py. Find validation, the store call and the 201 response. Which file should contain the database write?

3 Define a model before creating its table

A1 Complete the Task columns

Open task_api/models.py. Task already inherits db.Model and declares its table name and integer primary key. In A1 replace pass with the two mapped columns below. Keep the supplied constraints.

Attribute	Mapping to declare	Rule
title	db.String(120), nullable=False	Required text
done	db.Boolean, nullable=False, default=False	Required Boolean with ORM insertion default False

Follow the existing id declaration as a syntax pattern. A model describes mapping. Task(...) constructs a Python object; defining the class does not insert a row or create a table. The ORM supplies the keyword argument constructor.

Check your work

```
python -m unittest checks.ModelChecks -v
```

Expected: one check passes. Now initialize the working database and record its absolute path:

```
python -m flask --app task_api init-db
python -m flask --app task_api db-info
```

Expected: schema version 1 and zero tasks in instance/task_manager.db. This supplied init-db command applies the initial teaching schema. Later updates use upgrade-db. Do not replace or delete this file during the lab.

Observe a separate disposable experiment

```
python lifecycle_demo.py
```

This supplied script uses an in-memory database for the experiment only. Predict the id after construction, the id after flush and the row counts after rollback and after commit. It does not alter your working file.

Question 2. Why can an object have an id after flush even when its inserted row is later rolled back? Does entering app.app_context() commit a transaction?

4 Represent and create a task

A2 Build the base public dictionary

In Task.to_dict complete A2 by assigning a dictionary to the variable data. Its keys are id, title and done, and their values come from self. Keep the final return data statement supplied at the end of the method. Leave later B1 and D1 regions unchanged for now.

```
# Generic dictionary syntax
example = {"label": self.label}
```

Serialization selects public fields. It does not add or commit an object. Later parts will add entries to this same data dictionary rather than replace your earlier method.

Check your work

```
python -m unittest checks.RepresentationChecks -v
```

Expected: one check passes.

A3 Create a persistent task

In store.py complete TaskStore.create. fields has already been validated. Construct a Task from title and the optional done value, using False when done is omitted. Register the object with db.session.add, call the supplied self._commit helper and return its public dictionary after success.

Check your work

```
python -m unittest checks.CreateChecks -v
```

Expected: three checks pass. The route returns 201 and Location after the commit succeeds. The supplied get method reads one task and lets the route return 404 when absent.

Start the server and create T1

```
# Terminal A
python -m flask --app task_api run --debug
# Terminal B
python api_client.py GET /api/health
python api_client.py POST /api/tasks --json payloads/task.json
```

Health identifies task_manager. Record the actual returned Location as T1 and its id, title and done value. You will keep this same T1 for the relationship work. Do not assume the id is 1.

5 Read and update the original tasks

A4 Complete TaskStore list

Build db.select(Task), order by Task.id, execute the statement and collect model objects with scalars().all(). Build a list of their public dictionaries and return it. A read does not require a write commit.

Check your work

```
python -m unittest checks.ListChecks -v
```

Expected: one check passes. Send requests from Terminal B:

```
python api_client.py GET /api/tasks
python api_client.py POST /api/tasks --json payloads/second_task.json
```

Record the second actual URL and id as T2. Keep T1 and T2. In commands below, /api/tasks/1 is an illustrative path: replace 1 with your actual T1 id.

A5 Complete TaskStore update

Load the existing Task with db.session.get. If absent, raise ApiError with 404, not_found and Task not found. Change title or done only when its key is present in fields. Call self._commit and return the public dictionary. Keep the same loaded object.

The validator checks the entire payload before the store changes anything. An omitted title keeps its value. A supplied done=False must be applied.

Check your work

```
python -m unittest checks.UpdateChecks -v
```

Expected: three checks pass.

```
python api_client.py PATCH /api/tasks/1 --json payloads/complete.json
python api_client.py PATCH /api/tasks/1 --json payloads/reopen.json
python api_client.py PATCH /api/tasks/1 --json payloads/invalid_mixed.json
python api_client.py GET /api/tasks/1
```

Expect true, then false, then 400 for the invalid mixed body. The final read should keep the values saved before that rejected request. Question 3. Why is checking key presence different from testing fields.get("done") as a condition?

6 Delete a disposable task and check persistence

A6 Complete TaskStore delete

Load the Task. Return False if absent. Otherwise mark it with db.session.delete, call self._commit and return True. The supplied route returns 204 with an empty body on success and 404 on absence.

Check your work

```
python -m unittest checks.DeleteChecks -v
```

Expected: two checks pass. Create a third disposable task, T3. Delete that task and read its URL again. Keep T1 and T2 for the rest of the lab.

```
python api_client.py POST /api/tasks --json payloads/task.json
python api_client.py DELETE /api/tasks/3
python api_client.py GET /api/tasks/3
```

Replace 3 with T3's actual id. Expect 201, then 204, then 404.

Explain flush commit and rollback

```
python lifecycle_demo.py
python -m unittest checks.TransactionChecks -v
```

Expected experiment: None after construction; an integer id after flush; zero rows after rollback and one after the later commit. The two checks exercise transaction behavior and the supplied failure helper.

Question 4. Distinguish execute, flush and commit. Which call completes the write transaction? Why can rollback not undo a previous successful commit?

Save the first evidence and restart

Read T1 and T2, record their values and the db-info path, stop the server with Ctrl+C and restart from this same folder. Read the same URLs. Their saved values should still be available. Record this evidence before changing any model.

```
python -m unittest checks.SetupChecks checks.ModelChecks checks.RepresentationChecks -v
python -m unittest checks.CreateChecks checks.ListChecks -v
python -m unittest checks.UpdateChecks checks.DeleteChecks checks.TransactionChecks -v
```

After A1 to A6, these groups total 15 passing checks. They use temporary databases. Keep your A code; you will extend it in place.

7 Add projects to the existing model

B1 Map the one to many relationship

Lina now groups her existing tasks into projects. Stop the server before editing mappings. Work in the same models.py. A Project can have many Tasks; a Task can be unassigned or belong to one Project.

Location	Add	Required properties
Task B1_TASK	project_id column	Integer, ForeignKey("projects.id", ondelete="SET NULL"), nullable=True
Task B1_TASK	project relationship	Target "Project", back_populates="tasks"
B1_PROJECT	Project mapped class	Table projects; integer id PK; name String(100) not null
Inside Project	tasks relationship	Target "Task", back_populates="project", order_by="Task.id"
Inside Project	to_dict method	Return the project id and name
Task B1_PUBLIC	Public project key	Add project_id from self to the existing data dictionary

Use Task as your pattern for class inheritance, table name, primary key and serialization. Declare the Project class in B1_PROJECT. C1 will later add a scalar relationship inside this new class. The existing task columns and CRUD methods stay in place.

A foreign key stores an id in a column. db.relationship gives Python object navigation. back_populates names the attribute on the opposite class. It does not name a table.

Check your work

```
python -m unittest checks.ProjectModelChecks -v
```

Expected: one check passes against a temporary database built from your new mappings. It does not upgrade the working file. Do not restart the HTTP server yet.

Question 5. Which table stores the foreign key? Why is Task.project scalar while Project.tasks is a collection? What does nullable=True allow?

8 Upgrade the same database and keep T1

B1 Continue with schema version 2

The Python model now mentions project_id, but your existing tasks table was created in part A without that column. db.create_all() creates missing tables; it does not add a column to an existing table. Use the supplied incremental command while the server is stopped:

```
python -m flask --app task_api upgrade-db --to 2
python -m flask --app task_api db-info
```

The command creates projects and adds a nullable foreign key column to the existing tasks table. It keeps the original task ids, titles and done values. It also writes a backup under instance/backups before changing an initialized database.

Expected: schema version 2, the same absolute task_manager.db path and the same number of tasks. schema.py contains the supplied SQLite teaching updates; you do not need to implement or edit that infrastructure.

```
python -m flask --app task_api run --debug
# In Terminal B, replace these ids with your actual T1 and T2
python api_client.py GET /api/tasks/1
python api_client.py GET /api/tasks/2
```

Read your original T1 and T2, not newly recreated substitutes. Compare id, title and done with your part A evidence. Each now has project_id null. There is no need to recreate either task.

Create the new parent records

```
python api_client.py POST /api/projects --json payloads/project.json
python api_client.py POST /api/projects --json payloads/empty_project.json
```

Record the actual project ids as P1 and P2. The supplied project CRUD uses the class you added. Its GET response includes task_ids for navigation; before assignments those lists are empty.

Question 6. Why is changing models.py insufficient to change an existing table? Which values on your original T1 demonstrate that this was an upgrade of the same data?

9 Assign move and unassign an original task

B2 Complete TaskStore assign

In store.py, Project = related_model("Project") is supplied before B2. This helper finds the class you added or reports that its model stage is unfinished. require(Model, id) is another supplied helper: it returns the object or raises ApiError 404.

- Load the Task. If a project id is supplied, load the Project before changing the Task.
- Set task.project to that object, or to None to remove an assignment.
- Commit with self._commit and return the Task dictionary. Keep its id, title and done.

Check your work

```
python -m unittest checks.ProjectChecks -v
```

Expected: three checks pass. A target that does not exist must leave an earlier assignment unchanged.

Edit payloads/assign_project.json to contain your actual P1 id. Replace every illustrative path id below with the real task or project id:

```
python api_client.py PUT /api/tasks/1/project --json payloads/assign_project.json
python api_client.py GET /api/tasks/1
python api_client.py GET /api/projects/1
```

- Assign the original T1 to P1. Compare its project_id with P1's task_ids.
- Edit the payload to P2, repeat the PUT and read both projects. T1 moves between their collections.
- Use payloads/unassign_project.json to set JSON null, which becomes Python None.
- Restore P1. Try a project id you confirmed is absent using GET /api/projects/{id}; expect 404 and the P1 assignment retained.
- Leave T1 in P1 and T2 unassigned for the later reports.

Question 7. Why load the parent before assigning? Which SQL column changes when you assign through task.project? Explain why a new Task is not needed.

10 Add one optional project details record

C1 Extend the existing Project class

Continue in the same application. Stop the server. A Project may have one details record or none. Each existing details row must belong to one project. The new unique foreign key imposes the maximum of one.

Location	Add	Required mapping
Inside your Project class	details relationship	"ProjectDetails", back_populates="project", uselist=False, cascade="all, delete-orphan", single_parent=True
C1_DETAILS	ProjectDetails class	Table project_details; integer id PK
Inside ProjectDetails	project_id column	Integer FK to projects.id, ondelete="CASCADE", nullable=False, unique=True
Inside ProjectDetails	description column	String(500), nullable=False, default=""
Inside ProjectDetails	project relationship	"Project", back_populates="details"
Inside ProjectDetails	to_dict method	Return id, project_id and description

Place details inside the Project class you wrote in B1. Keep its tasks relationship. Place the new ProjectDetails class in C1_DETAILS. uselist=False makes navigation scalar; unique=True is the database constraint. The supplied cascade treats details as owned by its project.

Check your work

```
python -m unittest checks.DetailsModelChecks -v
```

Expected: one check passes. Now extend the same working database:

```
python -m flask --app task_api upgrade-db --to 3
python -m flask --app task_api db-info
python -m flask --app task_api run --debug
```

Expected: version 3 and the same task and project records. Read the original T1 and P1; their assignment must remain. Question 8. How can ProjectDetails.project_id be required while a Project can still have no details?

11 Create then update the same details object

C2 Complete ProjectStore put_details

The supplied method prefix makes Project and ProjectDetails available. Load the Project. Store a Boolean flag indicating whether project.details is currently None.

- If absent, construct ProjectDetails with the validated description and connect it through project.details.
- If present, update the existing object's description rather than create a replacement row.
- Commit and return two values: the details dictionary and the created flag. Python syntax for two returned values is return first_value, second_value.

The route uses that flag for 201 on creation and 200 on a later update. Keep the details id across both requests.

Check your work

```
python -m unittest checks.DetailsChecks -v
```

Expected: two checks pass. They check the two requests and database rejection of a duplicate details row.

```
python api_client.py PUT /api/projects/1/details --json payloads/details.json
python api_client.py PUT /api/projects/1/details --json payloads/details_update.json
python api_client.py GET /api/projects/1
```

Use your actual P1 id. Record 201 then 200, the same details id and the changed description. Also read T1 to show that adding details did not replace the existing task.

Question 9. Why does uselist=False not replace UNIQUE? Why is updating the existing details object different from always inserting a new object?

Predict the owned record policy

The supplied project deletion operation keeps Tasks by unassigning them and removes the owned details record through the configured cascade. Shared Tags, introduced next, have a different role. You will inspect this behavior in the final checks; deleting P1 is not needed for the main story.

12 Add shared tags to the same tasks

D1 Define the many to many mapping

Stop the server. A Task can have several Tags and a Tag can describe several Tasks. Add one association table and the following mappings in models.py.

Region or class	Add	Required mapping
D1_TABLE	task_tags db.Table	Name task_tags; task_id and tag_id are integer FKs and together the PK
Association FKs	Reference targets	tasks.id and tags.id; ondelete="CASCADE" on both
Task D1_TASK	tags collection	"Tag", secondary=task_tags, back_populates="tasks", order_by="Tag.id"
D1_TAG	Tag class	Table tags; integer id PK; name String(40), non-null and unique
Inside Tag	tasks collection	"Task", secondary=task_tags, back_populates="tags"
Inside Tag	to_dict method	Return id and name
Task D1_PUBLIC	tag_ids public field	Build a list of tag ids from self.tags and add it to data

Place task_tags at the top of the module in D1_TABLE so Task and Tag can refer to it. Each association column is marked primary_key=True. That declares a composite key, not two independent primary keys. The same pair can be stored once.

Keep id, title, done and project_id in the Task representation. A normal for loop and append is suitable for tag_ids; PYTHON_GUIDE.md explains both.

Check your work

```
python -m unittest checks.TagModelChecks -v
```

Expected: one check passes. Extend the working file, then restart:

```
python -m flask --app task_api upgrade-db --to 4
python -m flask --app task_api db-info
python -m flask --app task_api run --debug
```

Read the original T1 and P1 again. Its assignment and details remain, and T1 initially has tag_ids []. Question 10. Which table stores the links, and why is neither Task nor Tag the sole parent of the shared association?

13 Attach and detach a shared tag

D2 Complete TaskStore attach_tag

The Tag class is made available by the supplied prefix. Load the existing Task and Tag with require. If the Tag is not in task.tags, append it. Commit and return the Task dictionary. Reuse the existing Tag rather than construct another one.

D3 Complete TaskStore detach_tag

Load both objects. If their association exists, remove the Tag from task.tags. Commit and return no value; the route returns 204. In this API a repeated removal between two existing entities still returns 204.

Check your work

```
python -m unittest checks.TagChecks -v
```

Expected: three checks pass. They check repeated attachment and sharing, removal of one link and database rejection of a duplicate pair.

```
python api_client.py POST /api/tags --json payloads/tag.json
```

Record G1. Use real T1, T2 and G1 ids in these illustrative paths. The association requests need no JSON body:

```
python api_client.py PUT /api/tasks/1/tags/1
python api_client.py PUT /api/tasks/1/tags/1
python api_client.py PUT /api/tasks/2/tags/1
python api_client.py DELETE /api/tasks/1/tags/1
python api_client.py GET /api/tasks/1
python api_client.py GET /api/tasks/2
python api_client.py GET /api/tags
```

After the sequence, T1 has no G1 link; T2 still has G1; G1 still exists. Your original tasks and saved values remain.

Question 11. Which task_tags row is removed? Why would db.session.delete(tag) be a different operation? Stop and restart the server, then read the same tasks, P1 details and tags again.

14 Read task and project values together

E1 Complete the related task report

Lina wants task titles with their project names, including unassigned tasks. Complete ReportStore.tasks_with_projects. Select Task.id, Task.title and Project.name labelled project_name. Start from Task, outer join Task.project and order by Task.id.

Execute the statement and collect rows with all(). Build dictionaries with exactly id, title and project_name using the matching row attributes. Do not use scalars() when you need all selected columns.

Query selection	Extraction	Each item
Task model	scalars().all()	Mapped Task object
Several selected columns	all()	Result row of selected values

```
python api_client.py GET /api/reports/tasks-with-projects
```

Use the original T1 assigned to P1 and T2 unassigned. Both must appear. T1 has P1's name; T2 has project_name null. Adjust assignments first if your current data differs.

Question 12. Write the equivalent SQL LEFT JOIN. What disappears with an inner join? In this three-column query, what would scalars() keep and what would it discard?

No schema update is needed for this report because it reads the existing tables. Complete E2 next, then run the two report checks.

15 Count tasks while retaining empty projects

E2 Complete the project count report

Start from Project so the report includes empty projects. Select Project.id, Project.name and func.count(Task.id) labelled task_count. Outer join Project.tasks, group by the two non-aggregate project columns and order by Project.id.

In ReportStore.project_counts execute the statement, collect rows with all() and return dictionaries containing exactly id, name and task_count. func is already imported. Keep task_count as an integer.

For an unmatched outer join row, COUNT(Task.id) ignores the NULL child id. COUNT(*) counts the retained row and would give one for an empty project in this query.

Check a result you can count yourself

Keep T1 in P1 and P2 empty. If your experiments added other tasks, use their actual assignments when checking the expected count.

Illustrative project state	Expected task_count
P1 contains only T1	1
P2 contains no tasks	0

```
python api_client.py GET /api/reports/project-counts
```

Check your work

```
python -m unittest checks.ReportChecks -v
```

Expected after E1 and E2: two checks pass. Their temporary test records do not need to share ids with your working records.

Question 13. Write the equivalent SQL GROUP BY query. Why do this report and the earlier task report start from different tables?

16 Save a complete project as one transaction

E3 Complete ProjectStore create_bundle

Read payloads/bundle.json and validate_bundle. The request contains a project name, one description and a list of tasks. The route validates every child before calling the store.

- Inside one try block construct and add the Project, then explicitly flush to observe the generated id within the uncommitted transaction.
- Create the connected ProjectDetails. Construct each Task with its validated fields, connect it to the Project and add it.
- Commit once after preparing all related rows.
- Catch SQLAlchemyError around the whole block, including flush. Roll back, then raise ApiError 500 with database_error and Database operation failed.
- After success return self.get(project.id). Keep the supplied prefix and imports.

Do not commit the parent separately. A later rollback cannot undo an earlier successful commit. The explicit flush practises the mechanism; it is not required before every ORM commit.

```
python api_client.py POST /api/project-bundles --json payloads/bundle.json
python api_client.py POST /api/project-bundles --json payloads/invalid_bundle.json
```

Expect 201 for the valid request and 400 for the invalid request before storage. The 400 tests validation. The isolated checker deliberately bypasses validation with a bad child to exercise a database failure after the parent flush.

Check your work

```
python -m unittest checks.BundleChecks -v
```

Expected: five checks pass, including rollback and subsequent valid work. Question 14. Where is the single commit boundary? Which rows from this operation should be absent after a failed child insert?

17 Demonstrate the complete evolving application

Verify the original records

- Use db-info to show schema version 4 and the same absolute task_manager.db path recorded in part A.
- Read your original T1 and T2 URLs. Compare their ids and core values with earlier evidence, allowing changes you explicitly made by PATCH.
- Show project navigation, one updated details record and a tag shared across tasks.
- Stop and restart the server. Read the same resources and compare their saved values and links.

Run the final checks

```
python -m unittest checks.SchemaChecks -v
python -m unittest checks -v
```

Expected: three schema checks, then 36 checks in the full suite. SchemaChecks checks preservation through upgrades, repeat and downgrade behavior, and rollback of a failed schema update. The full suite uses temporary databases; your request and restart evidence concerns the working file.

Submit

Submit this one project's source, EVIDENCE_TEMPLATE.md and named-check output. Exclude .venv, __pycache__ and instance. Keep your working database and automatic backups locally for demonstration. Do not submit a replacement project containing a fresh set of tasks as evidence of the upgrade.

Common problems

Symptom	First action
501	Complete the named method or required model stage
Missing project_id column	Stop the server and apply the matching upgrade after B1
Other missing related table	Check the C1 or D1 mapping and target schema version
Module not found	Check the activated interpreter and package installation
400 or 415	Read the JSON error and check types and the payload file

Primary references: Flask SQLAlchemy Quick Start and SQLAlchemy Basic Relationship Patterns.

https://flask-sqlalchemy.palletsprojects.com/en/stable/quickstart/
https://docs.sqlalchemy.org/en/20/orm/basic_relationships.html